

迈向通用具身操作

——训练时计算与推理时计算

作者：张楚斌、马世琳、郭昊仁

时间：2026/08

目录

Part 1 背景

Part 2 训练时计算

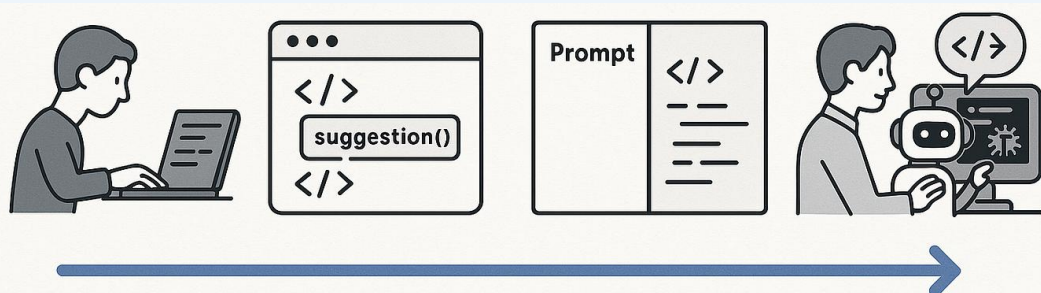
Part 3 推理时计算

Part 4 Safe-Pruner+HiFloat8

具身智能的突破会将智能的红利带到物理世界

大模型

大模型验证了 **scaling law** 的有效性，在**信息收集与处理**上超过人类，革新**信息世界**



MANUAL
CODING ERA

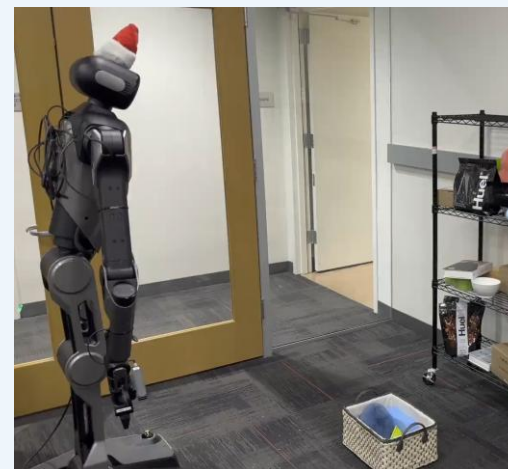
AI
AUTOCOMPLETE

NON-CONTEXTUAL
GEN CODE

AI CODING
AGENTS

具身智能

具身智能旨在把大模型的智能从信息处理扩展到**物理交互**，造福**物理世界**

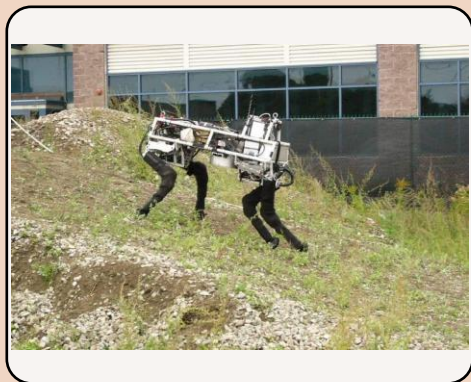


具身智能的目标是智能与通用



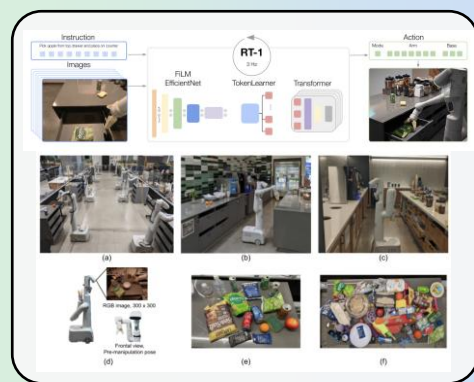
1969年，斯坦福设计了第一款六运动轴机械臂，通过专用规则进行控制

1969



2008年，波士顿动力的机器狗实现了复杂地形下的动态平衡与运动控制

2007



2022年谷歌开发的RT-1模型实现了端到端的通用机器人控制

2022



2026年英伟达发布的工作验证了基于人类视频数据的物理 scaling law

2026

本体与控制

智能与通用

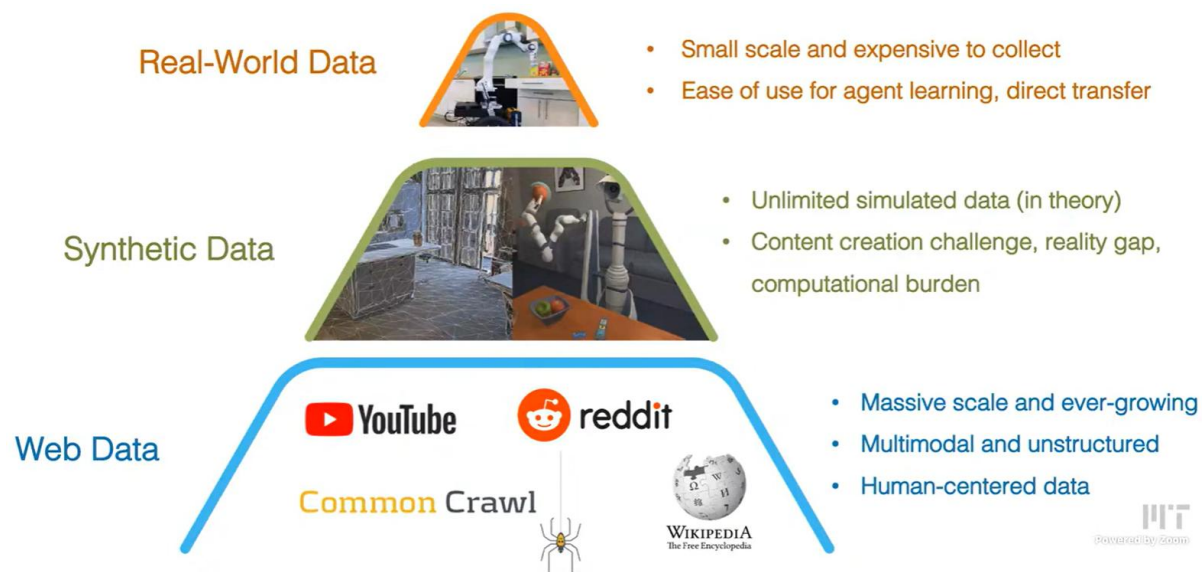


清华大学
i-VisionGroup

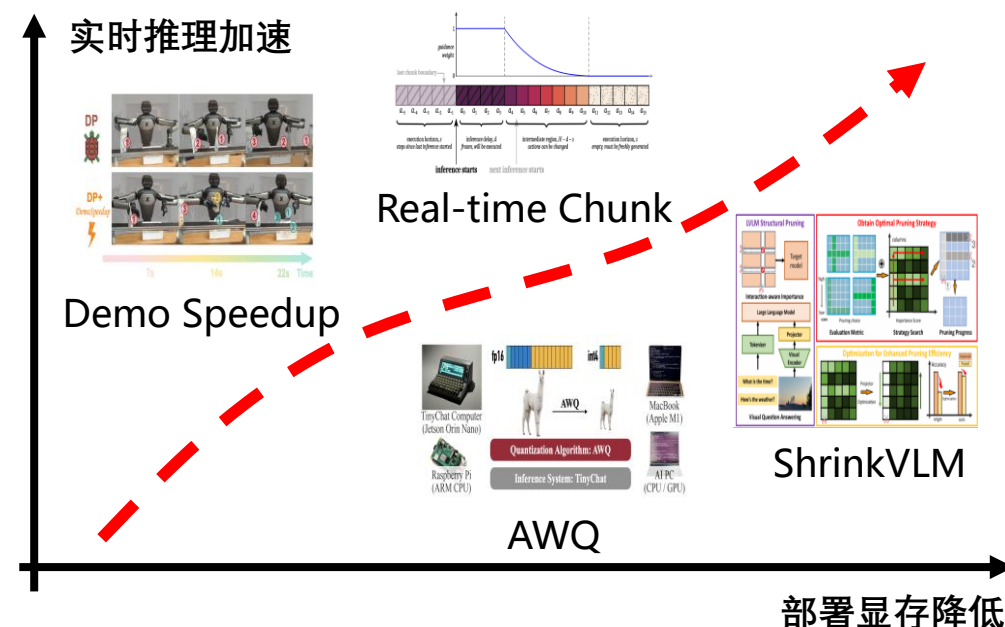
CANN

通用具身操作的挑战在于如何实现高效的**训练和推理时计算**

- “训练时计算” 研究在预训练阶段异构数据学习与利用
- “推理时计算” 研究在模型部署阶段的高效实时推理



具身操作预训练的数据来源多样，
需要统一建模学习目标



具身操作推理时计算包含与真实场景的
动态交互，需要**高效实时推理**



清华大学
i-VisionGroup

CANN

目录

Part 1 背景

Part 2 训练时计算

Part 3 推理时计算

Part 4 Safe-Pruner+HiFloat8

训练时，**世界模型**是通用具身操作的关键之一

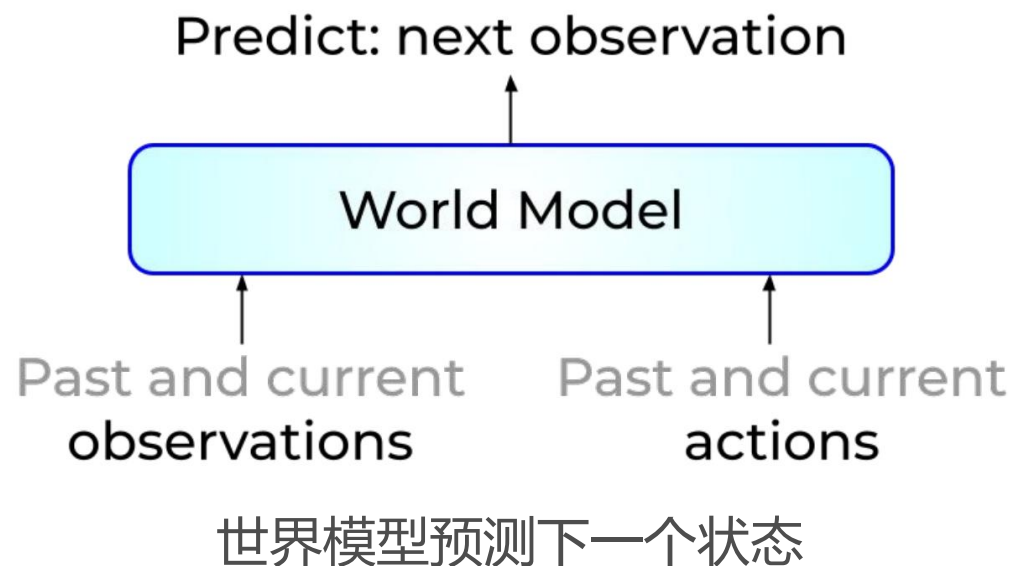
- 语言模型基于**下一 token 预测**任务实现了训练时计算的扩展
- 训练**世界模型**来基于动作条件预测**下一个状态**与训练语言模型来预测下一个 token 的预训练任务非常相似

Enter text:

One, two, three, four, mango

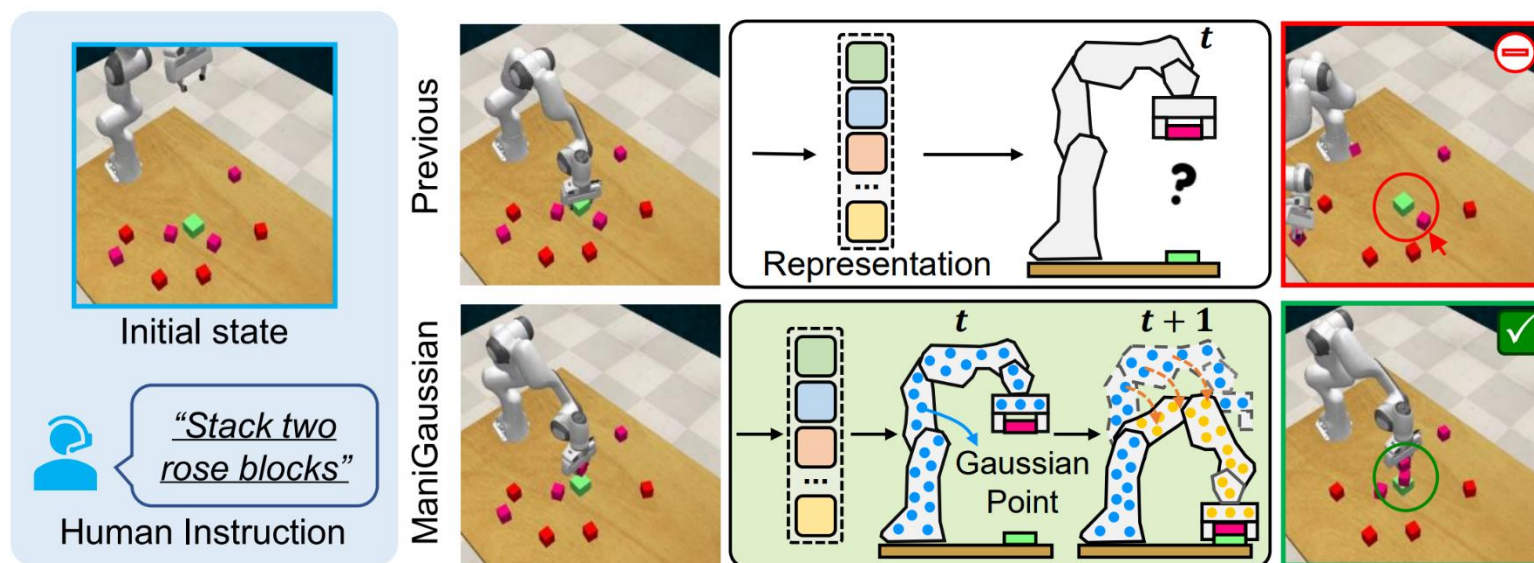
One , two , three , four , mango

语言模型预测下一个 token



ManiGaussian: 基于三维世界模型的机器人操作策略

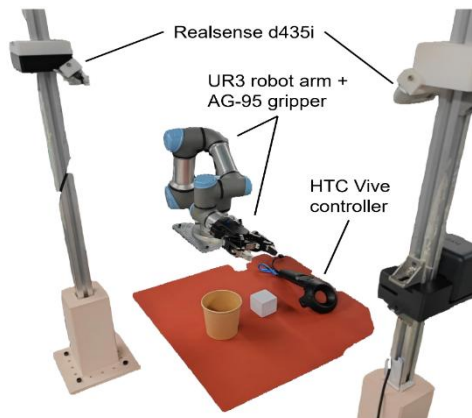
- 利用**三维高斯泼溅**表征三维场景的**几何先验信息**
- 利用**具身世界模型**表征三维场景的**动力学先验信息**



环境几何先验信息:
生成式3D Gaussian Splatting技术

环境动力学先验信息:
具身世界模型

真实机械臂部署：世界模型建模帮助策略高效学习真实任务

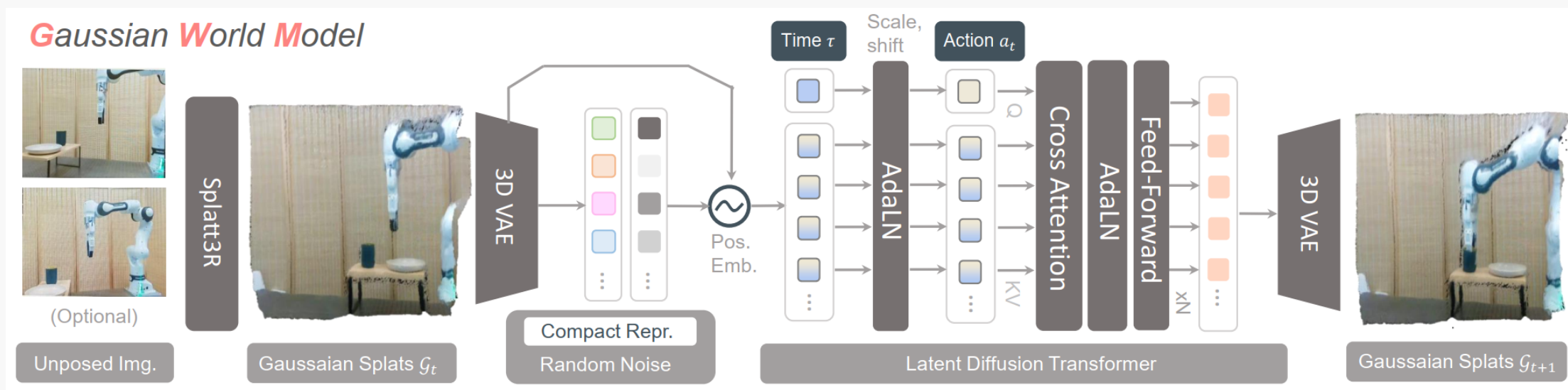


模仿学习训练（每个任务仅需~**10条**人类示教数据）



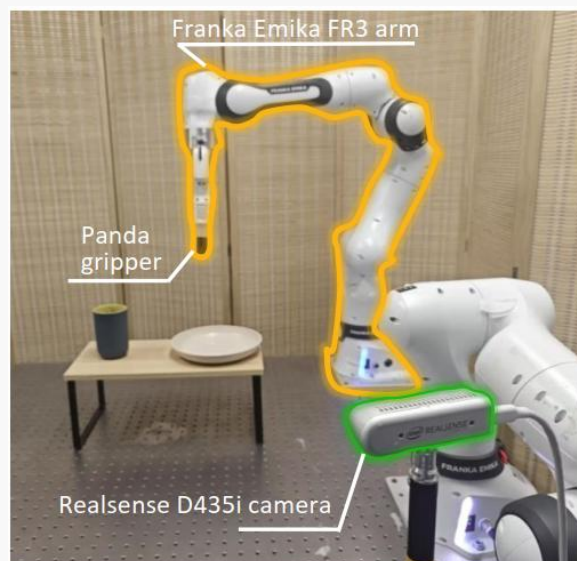
Gaussian World Model (GWM): 同时支持模仿学习、强化学习和视频生成的可扩展高斯世界模型

- **模型结构:** GWM 的核心是 Diffusion Transformer(DIT)
- **训练方式:** GWM 通过可泛化的三维模型把数据提升到三维来训练

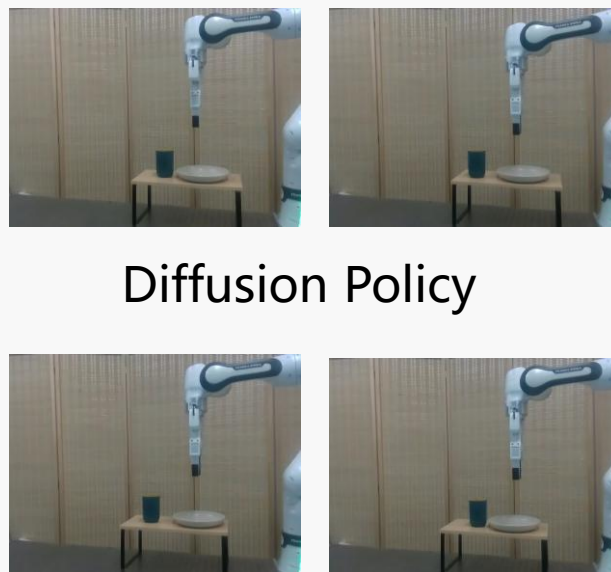
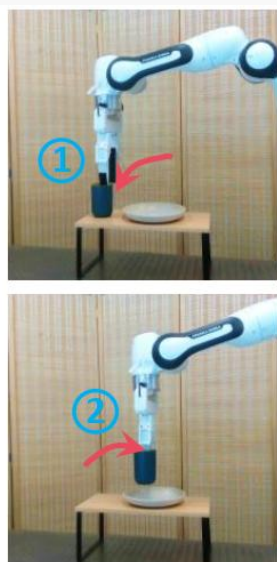


Gaussian World Model (GWM): 同时支持模仿学习、强化学习和视频生成的可扩展高斯世界模型

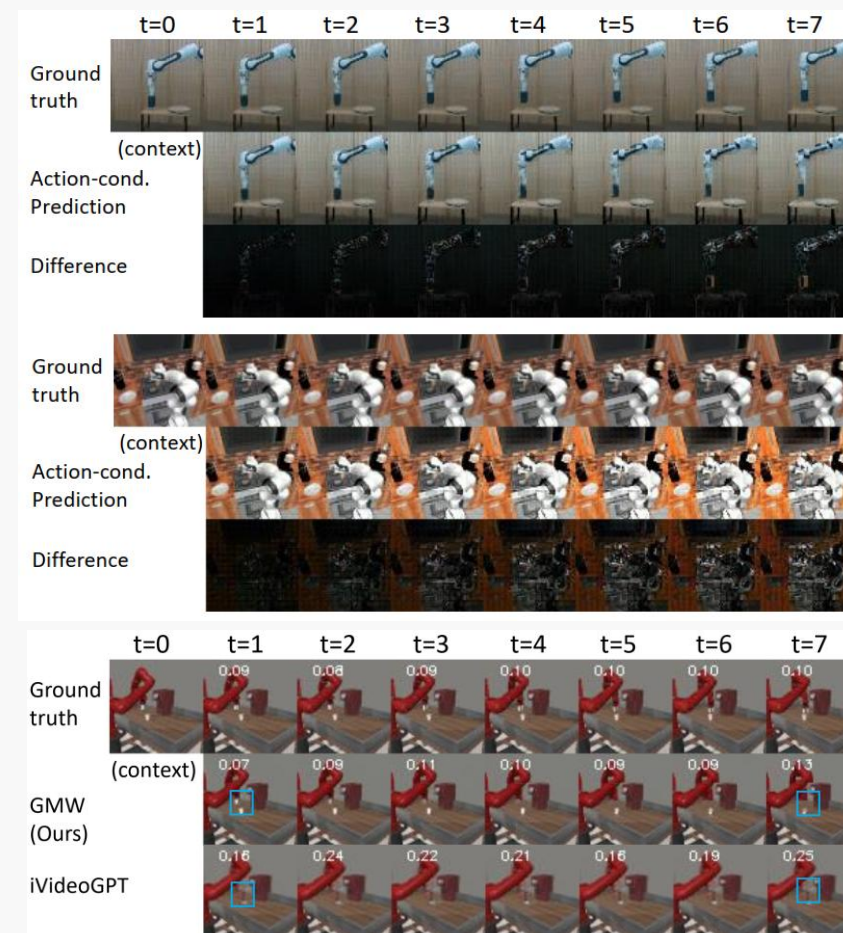
- GWM 可作为动作**条件视频生成模型**
- GWM 可以帮助 Diffusion Policy 少样本学习
需要理解物理变化的真实具身操作任务



真机实验环境



Diffusion Policy w/ GWM

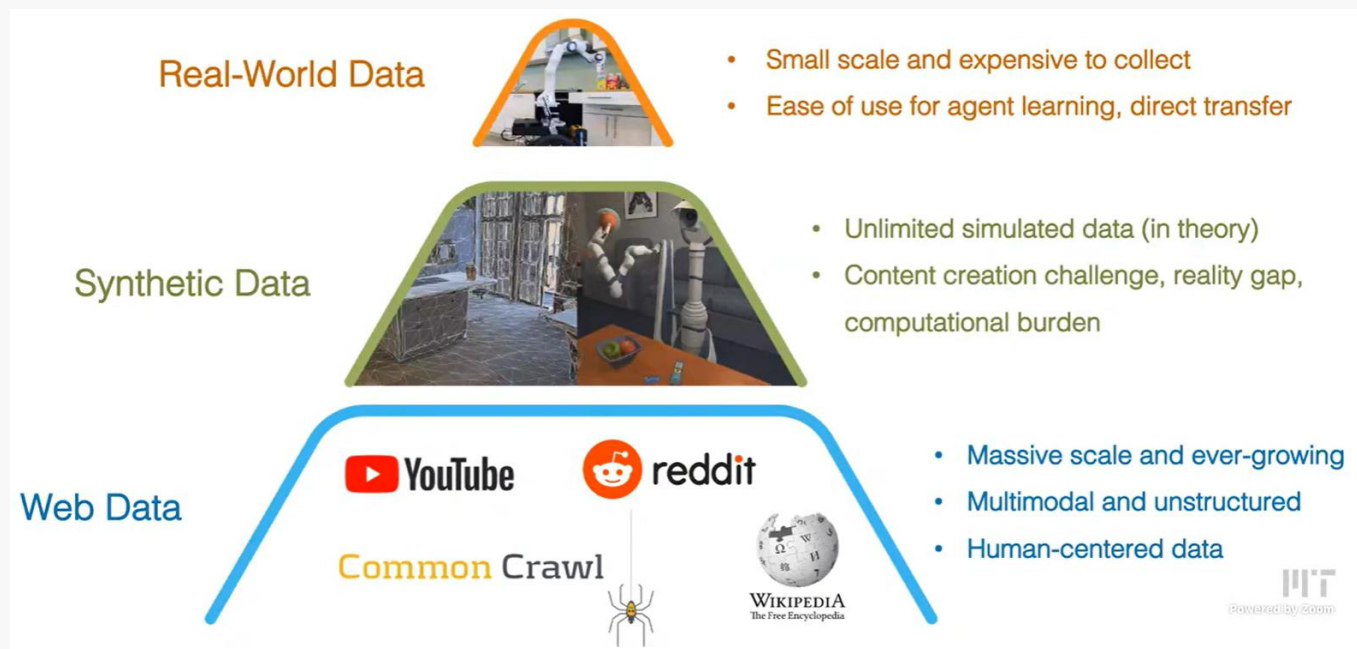


清华大学
i-VisionGroup

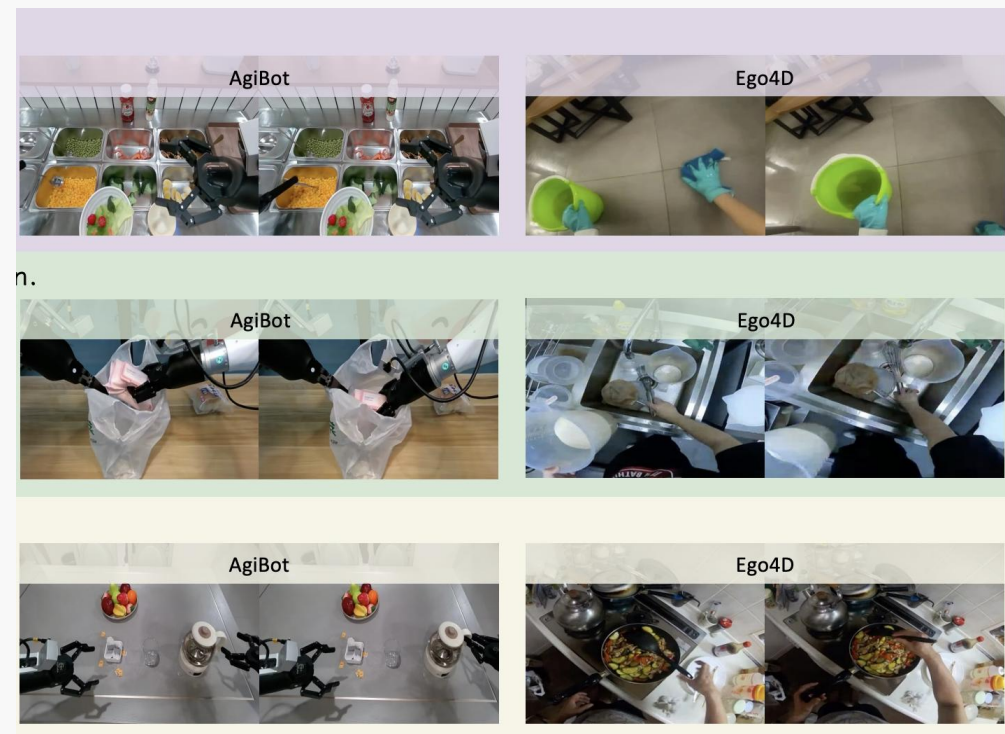
CANN

ManiGaussian 和 GWM 在三维建模上显式地学习了世界模型，但缺乏在**大规模数据**上的可扩展性

- 其中**人类数据**展现出巨大的优势：**采集轻量、反馈直接、可迁移**



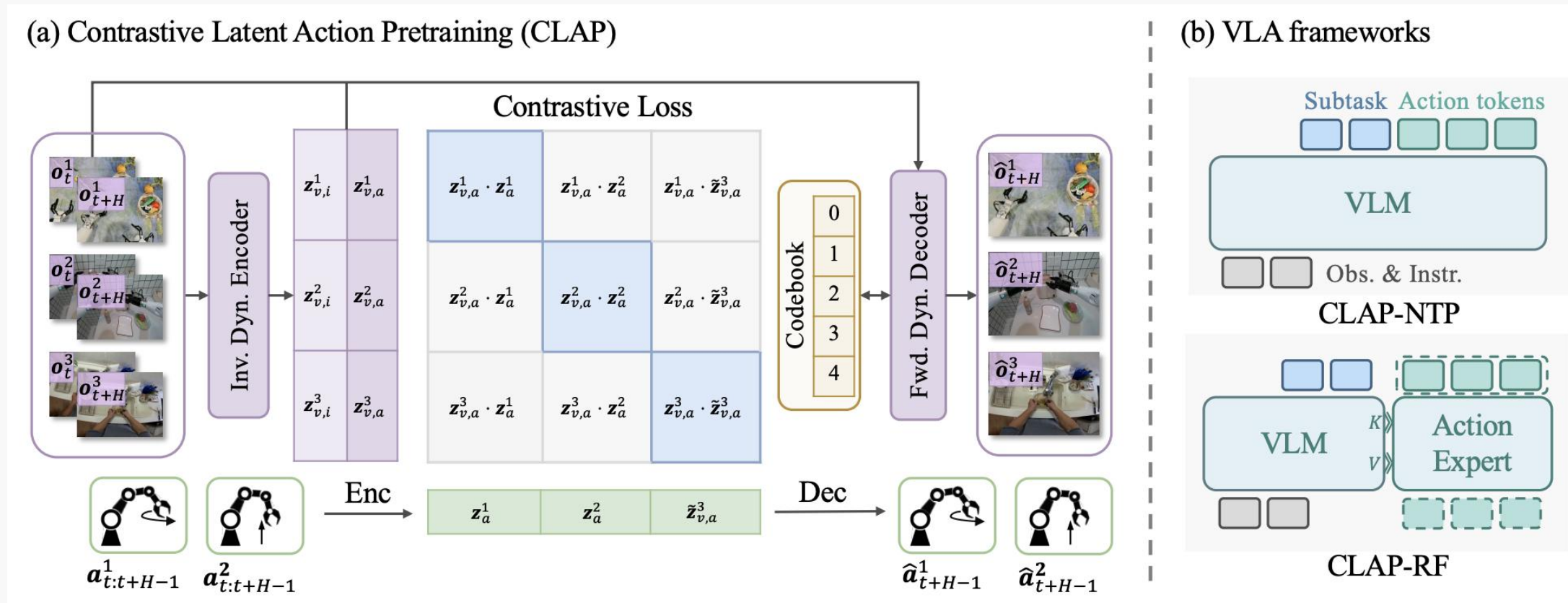
数据金字塔



可迁移性

利用人类视频训练 VLA

- 基于**世界模型**的建模提取**隐空间动作**
- 基于**对比学习**优化隐空间动作：对齐**视觉变化**和**物理可执行动作**



利用人类视频训练 VLA

- 基于世界模型的建模提取隐空间动作
- 基于对比学习优化隐空间动作：对齐视觉变化和物理可执行动作



Pick and place



Pack the doll



Fold the cloth

Method	<i>Pick and Place</i>		<i>PnP (OOD)</i>		<i>Pack the Doll</i>		<i>Fold T-shirt</i>	<i>Make Bouquets</i>		<i>Task Mean</i>
	<i>Pick (%)</i>	<i>Place (%)</i>	<i>Pick (%)</i>	<i>Place (%)</i>	<i>P&P (%)</i>	<i>Close (%)</i>	<i>Succ. (%)</i>	<i>C-1 (%)</i>	<i>C-2 (%)</i>	
π_0 [37]	85	75	65	60	<u>80</u>	<u>60</u>	<u>40</u>	40	<u>30</u>	54.0
$\pi_{0.5}$ [38]	<u>90</u>	<u>80</u>	<u>80</u>	<u>75</u>	<u>80</u>	<u>60</u>	50	<u>30</u>	40	<u>60.0</u>
UniVLA [10]	75	60	65	50	70	30	10	<u>30</u>	20	35.0
CLAP-NTP	<u>90</u>	<u>80</u>	85	85	<u>80</u>	70	30	<u>30</u>	<u>40</u>	58.7
CLAP-RF	95	85	85	70	90	70	<u>40</u>	40	50	62.7

利用人类视频训练 VLA

- 利用人类数据学习物品泛化

*"Make a bouquet using the **orange tulip** and **red rose**."*

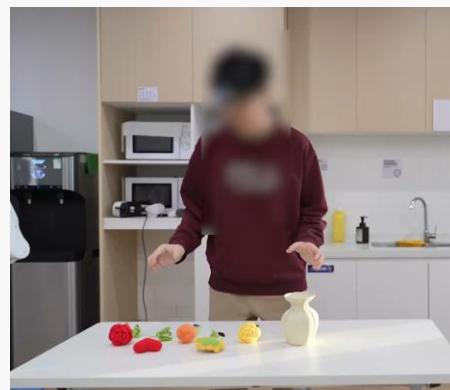


Unseen Object: **orange tulip**

Direct roll out
($\pi_{0.5}$)



Wrong pick!



Learn from
human videos

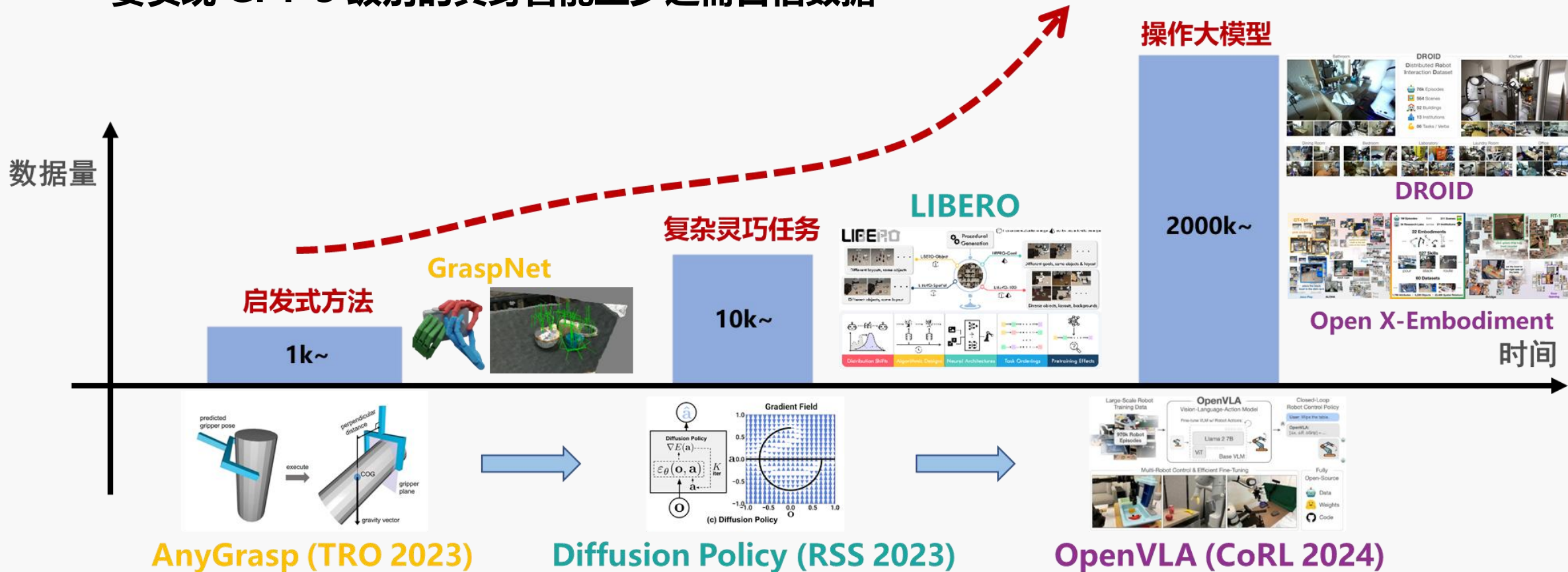


Success!



训练时计算：随着数据规模和模型架构的发展，具身操作范式从启发式转向模仿学习。对演示数据的采集和利用决定了模型上限

- 要实现 GPT-3 级别的具身智能至少还需百倍数据



目录

Part 1 背景

Part 2 训练时计算

Part 3 推理时计算

Part 4 Safe-Pruner+HiFloat8

推理时延会直接改变机器人执行轨迹

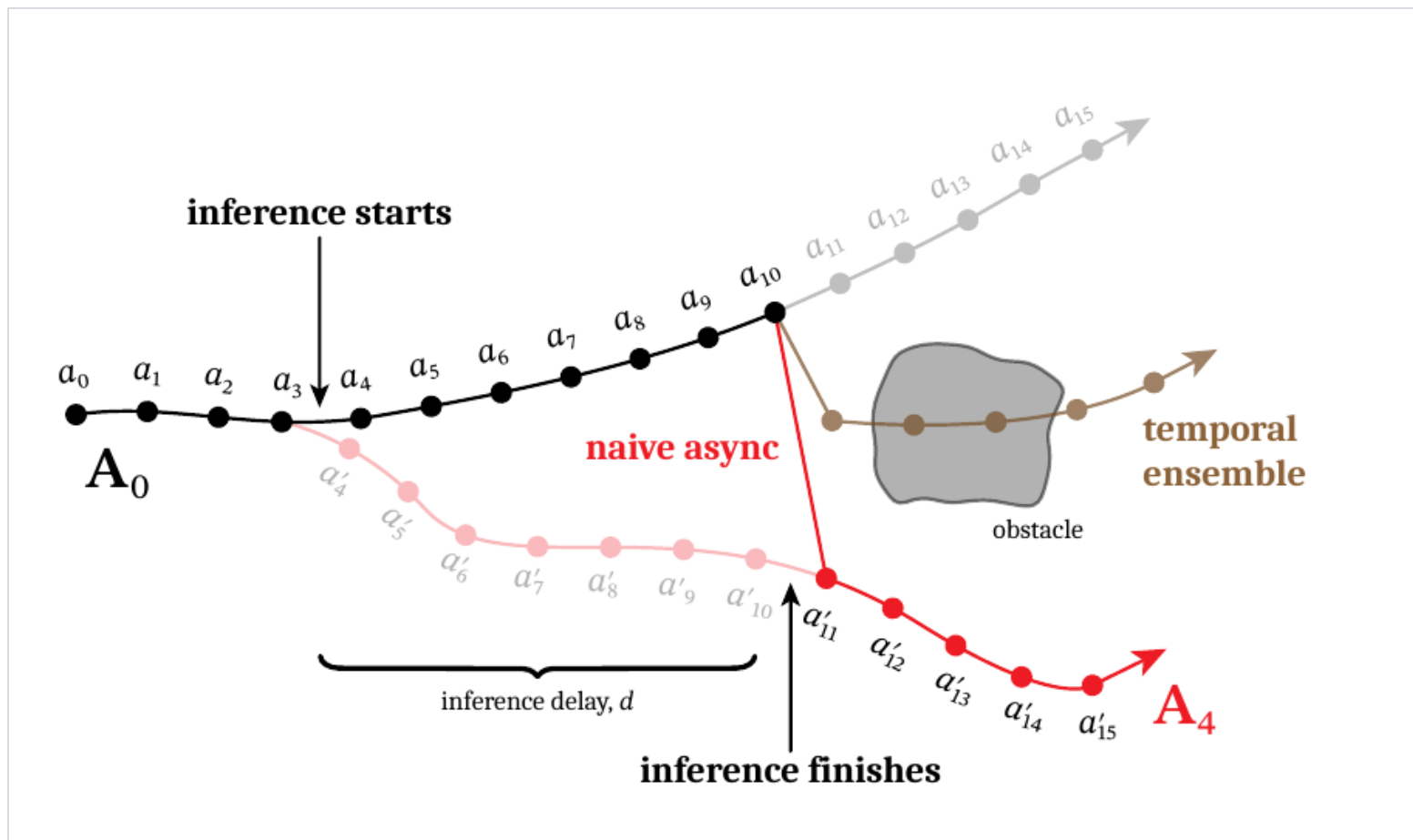
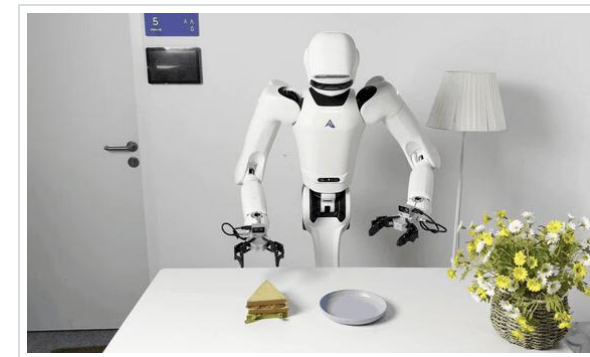


图: Black et al., Real-Time Execution of Action Chunking Flow Policies

<https://gitcode.com/cann>

REAL-TIME CONTROL



执行中的策略不会等待模型

- 新动作块生成时，机器人仍在执行旧轨迹
- 时延越长，新旧动作的起点偏差越大
- 在障碍附近，拼接误差会转化为碰撞风险

实时性是闭环策略的一部分：速度决定动作能否安全衔接。

SAFE-Pruner (ECCV 2026)

语义引导剪枝： 只保留动作相关 token

以任务语义为锚点，在速度与动作质量之间取得平衡。

01 语义引导

保留与任务相关的视觉 token

02 未来感知

预测深层注意力，避免浅层“短视”

03 即插即用

不改 VLA 权重，直接插拔

SAFE-Pruner: Semantic Attention-Guided Future-Aware Token Pruning for Efficient Vision-Language-Action Manipulation

Accepted to ECCV 2026

Shilin Ma¹ Chubin Zhang¹ Changyuan Wang¹ Yuji Wang¹ Yue Wu¹ Zixuan Wang¹ Jingqi Tian¹
Zheng Zhu² Yansong Tang^{1,†}

¹ Tsinghua Shenzhen International Graduate School, Tsinghua University ² GigaAI

[†] Corresponding author

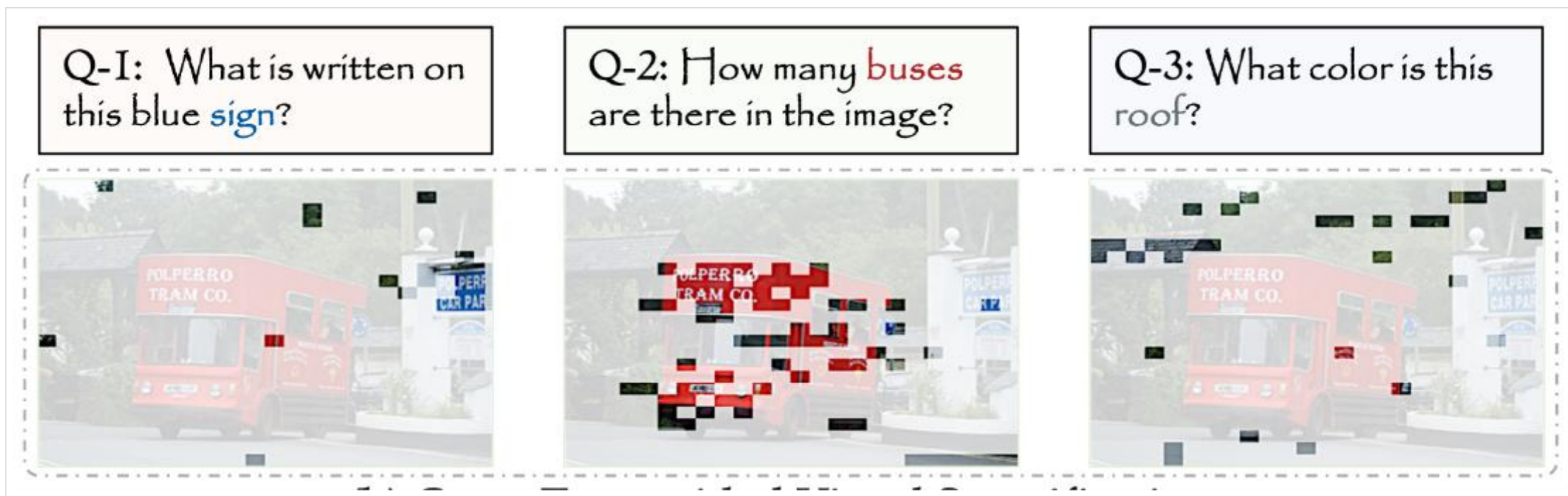


Paper

arXiv

Code

视觉token剪枝：只让任务相关的token参与注意力计算



N 个视觉 Token



重要性评分 / 重排



保留 $K \ll N$

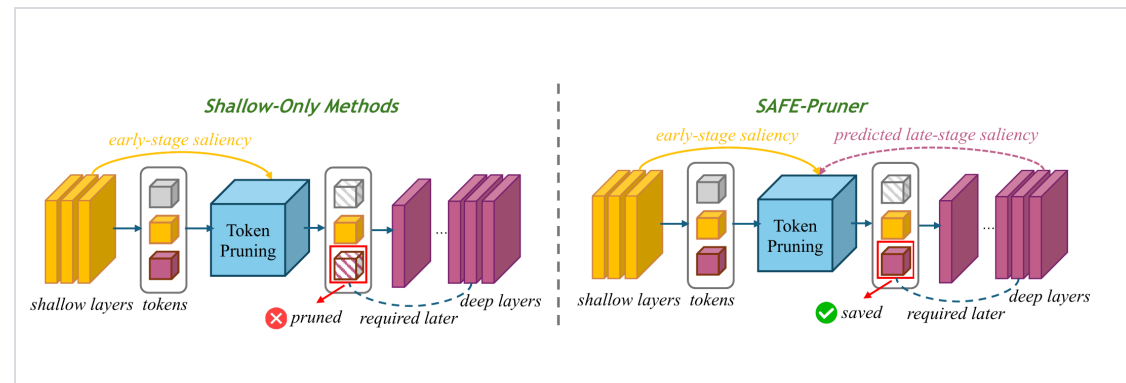
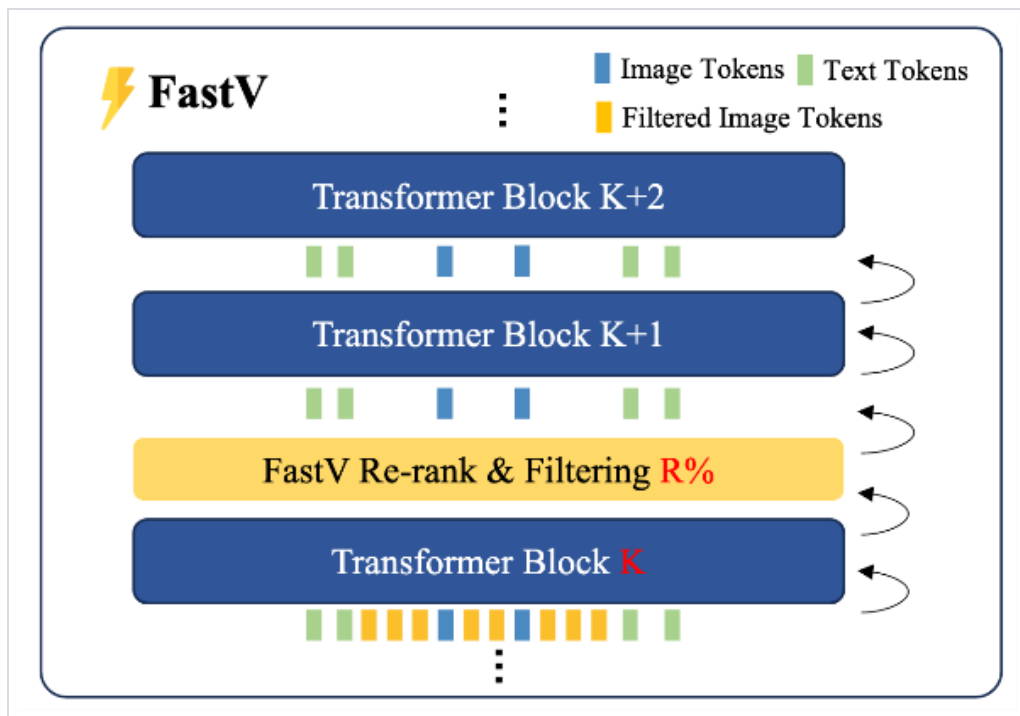


Transformer注意力计算更快

图：Zhang et al., SparseVLM: Visual Token Sparsification for Efficient Vision-Language Model Inference

<https://gitcode.com/cann>

现有token剪枝常在浅层决定：速度更快，却容易“短视”



浅层关注度低，并不等于深层不需要

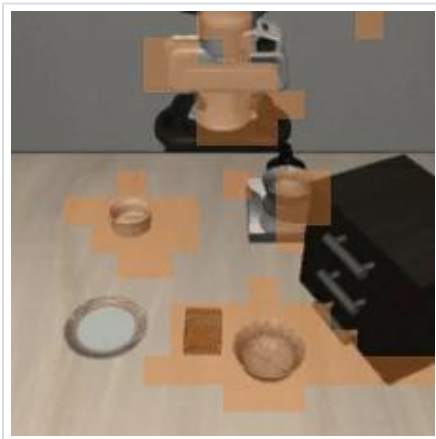
- 浅层注意力相对较为分散
- 深层注意力会逐步聚合到特定区域
- 越早剪枝，算力收益越大；判断失误的代价也越高

图：Chen et al., An Image is Worth 1/2 Tokens After Layer 2: Plug-and-Play Inference Acceleration for Large Vision-Language Models

FastV：在第 K 层按当前注意力重排并剪枝

在具身任务中，能否提前预测深层所关注的token？

具身智能任务中的语义注意力一致性



Frame 0



Frame 7



Frame 12



Frame 13



Frame 15

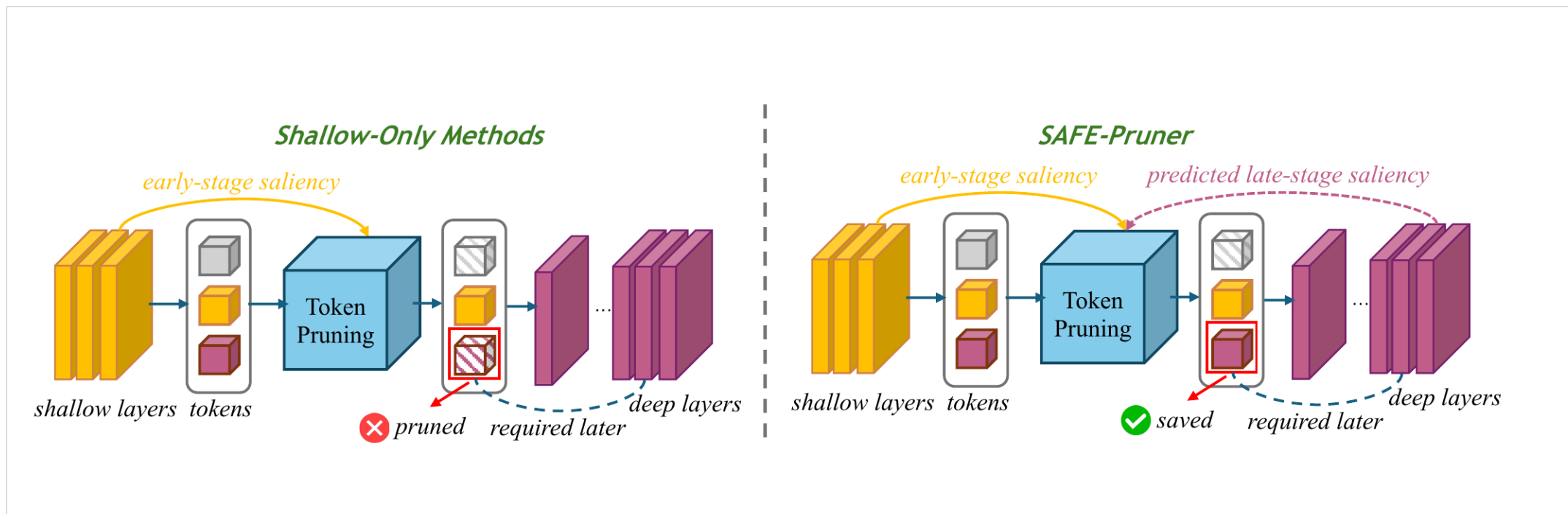
稳定阶段

相邻时间步持续关注同一语义实体

任务切换

注意力分布突变，触发参考帧刷新

SAFE-Pruner: 用历史深层注意力预测当前的“未来重要性”



传统方法

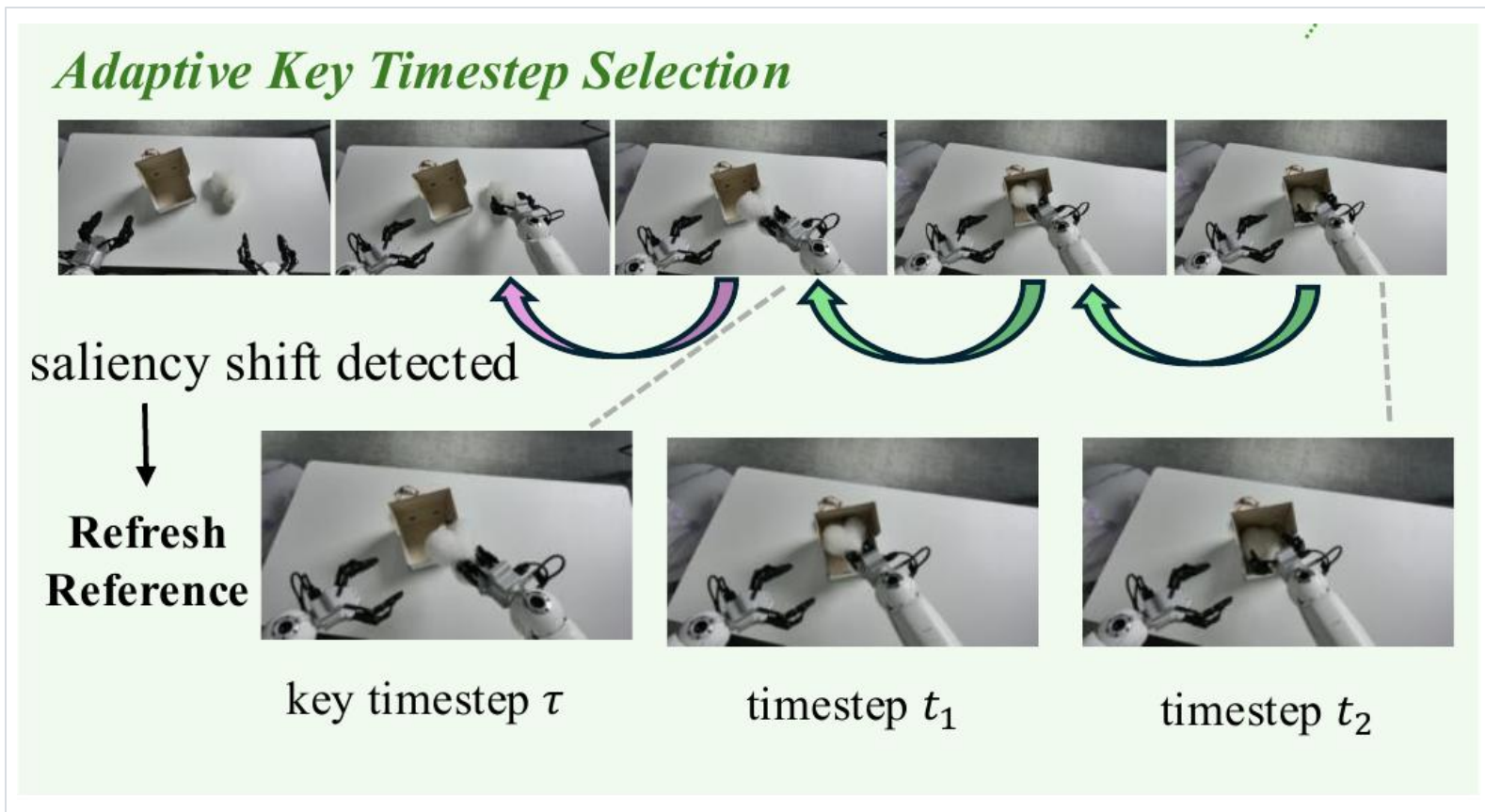
当前浅层注意力 → 立即剪枝

SAFE-Pruner

浅层分数 + 预测晚期分数 → 剪枝

训练自由 • 即插即用 • 同时适配自回归与流式 VLA

关键时间步选择：只在注意力模式变化时刷新深层参照



01 计算变化

比较当前浅层显著性与历史参照

02 检测突变

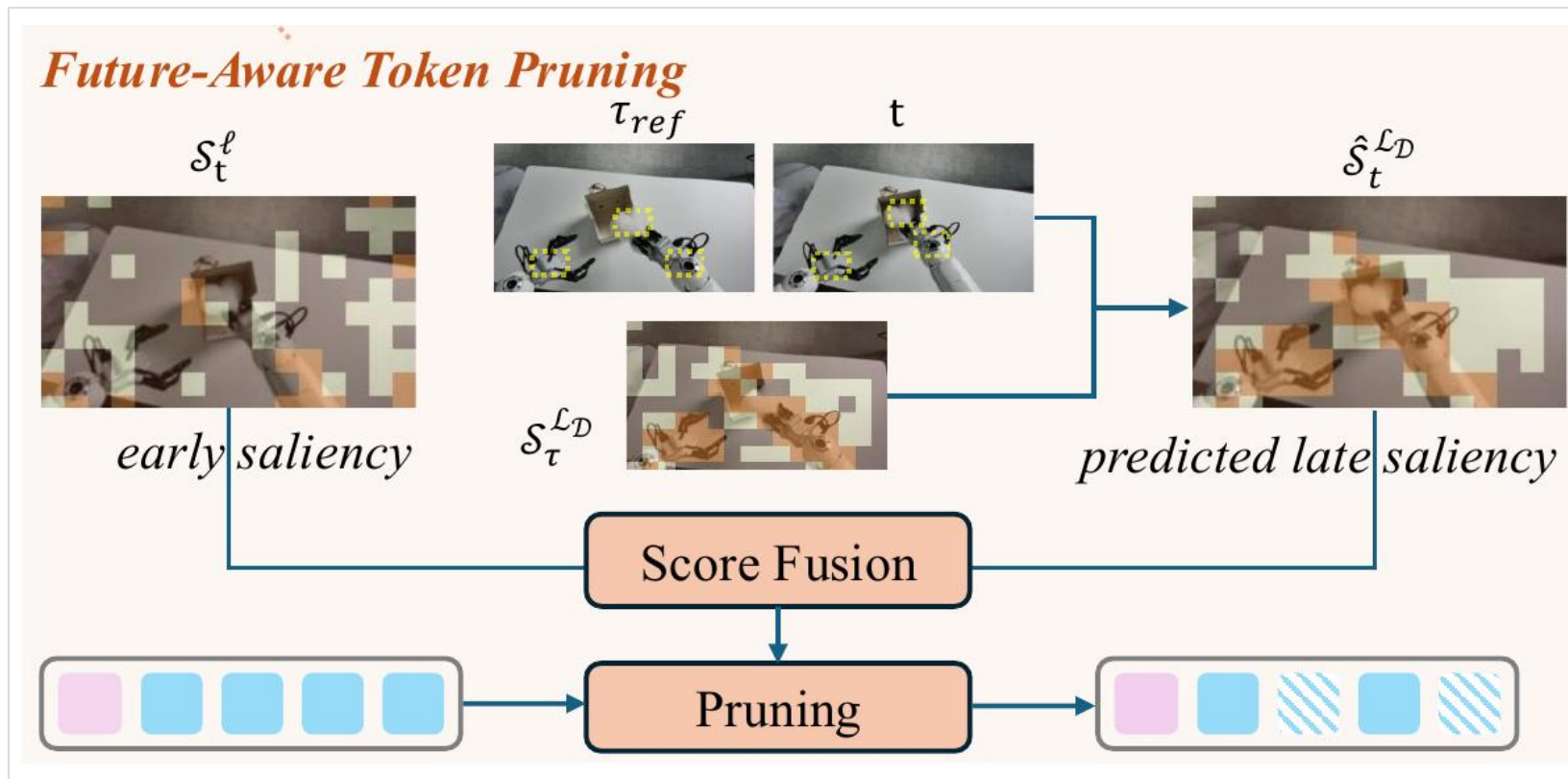
子任务交界处注意力模式快速切换

03 刷新参照

关键帧完整前向并缓存深层注意力

关键帧保真；非关键帧加速

未来感知剪枝：融合当前浅层分数与预测晚期分数



当前浅层显著性

$\mathcal{S}_t^l$

+

预测晚期显著性

$\hat{\mathcal{S}}_t^{\mathcal{L}_D}$

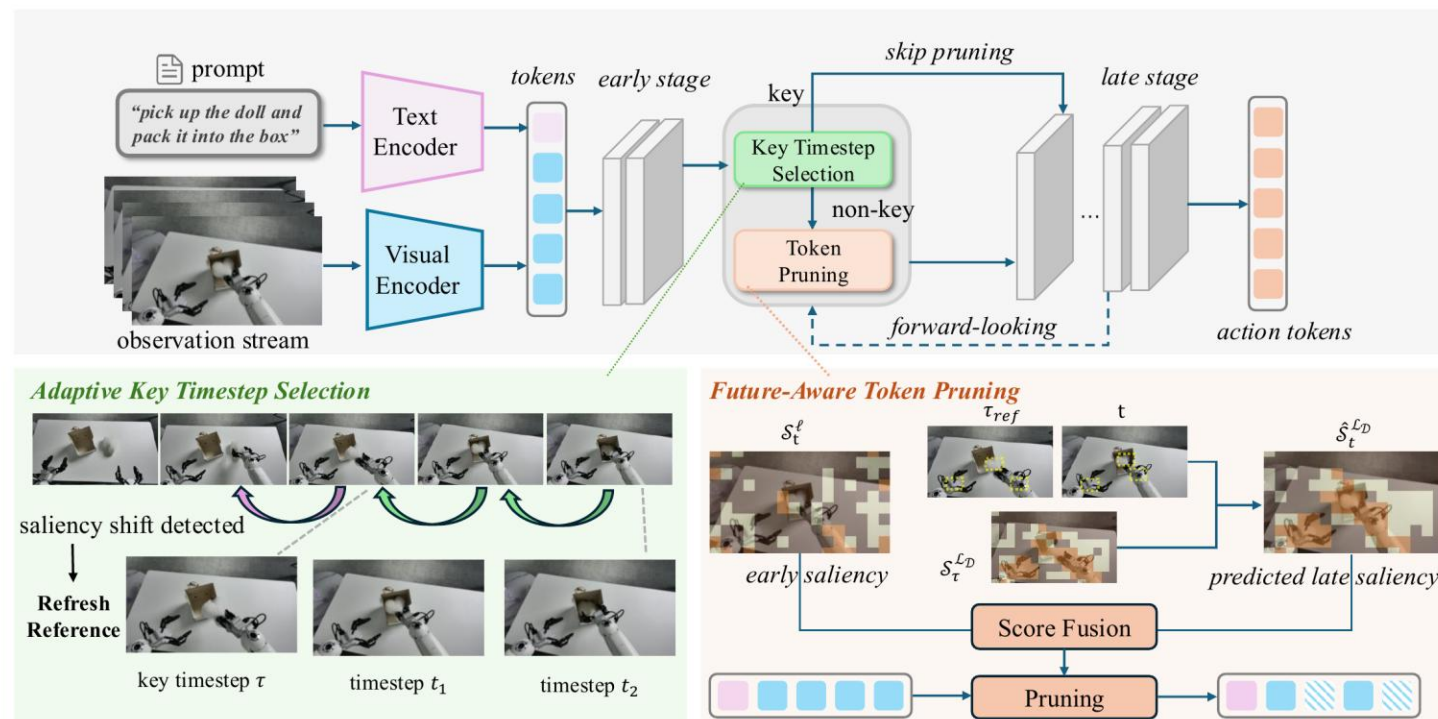
Score Fusion

↓

Top-K 保留

不训练预测器：直接复用历史深层注意力，开销极低。

完整流程：关键帧校准，非关键帧预测“未来显著性”



ONLINE INFERENCE

01 关键帧建立参考

晚期层完整推理，生成可靠的深层显著性

02 漂移触发刷新

浅层注意力发生突变时，自适应切换关键帧

03 非关键帧浅层剪枝

融合当前注意力与预测显著性，提前删去冗余 token

无需训练 · 即插即用 · 不修改 VLA 权重

全面比较：三类 VLA 设置中均实现更优效率—性能权衡

01 • LIBERO

Model	Method	Success Rate(%) ↑					FLOPs (T)↓	Latency (ms)↓
		Spatial	Object	Goal	Long	Average		
OpenVLA	Vanilla	84.6	86.6	78.2	53.2	75.7	1.862	48.24
	FastV	81.8	70.6	71.8	44.6	67.2	0.811	34.63
	SparseVLM	80.6	72.4	68.2	46.6	67.0	0.876	34.32
	DivPrune	78.6	62.8	66.2	43.2	62.7	0.832	37.18
	VLA-Cache	78.2	71.2	70.6	45.8	66.5	0.941	35.21
	VLA-Pruner	82.0	84.4	77.8	52.6	74.2	0.793	36.47
	Ours	82.2	84.8	77.6	52.0	74.2	0.742	32.18
OpenVLA-OFT	Vanilla	98.4	98.2	96.4	94.2	96.8	3.970	70.25
	FastV	97.8	92.8	95.6	91.8	94.5	2.141	50.24
	SparseVLM	98.6	91.8	94.8	92.0	94.3	2.289	52.15
	DivPrune	97.2	90.0	93.4	89.6	92.6	2.126	53.87
	VLA-Cache	97.0	92.2	94.6	91.0	93.7	2.730	58.14
	VLA-Pruner	97.4	93.0	94.0	92.0	94.1	2.234	54.53
	Ours	98.0	98.0	96.2	93.4	96.4	1.722	37.22
$\pi_{0.5}$	Vanilla	98.2	98.0	98.0	91.8	96.5	2.115	35.28
	FastV	96.4	96.8	95.6	89.0	94.5	1.612	28.64
	SparseVLM	94.6	97.2	94.8	89.2	94.0	1.713	29.03
	DivPrune	93.8	96.6	92.4	88.0	92.7	1.576	30.55
	VLA-Cache	94.8	96.0	96.4	88.2	93.9	1.632	28.14
	VLA-Pruner	95.8	97.2	95.6	88.0	94.2	1.583	32.64
	Ours	97.0	98.8	96.0	90.2	95.5	1.482	24.33

02 • SIMPLER

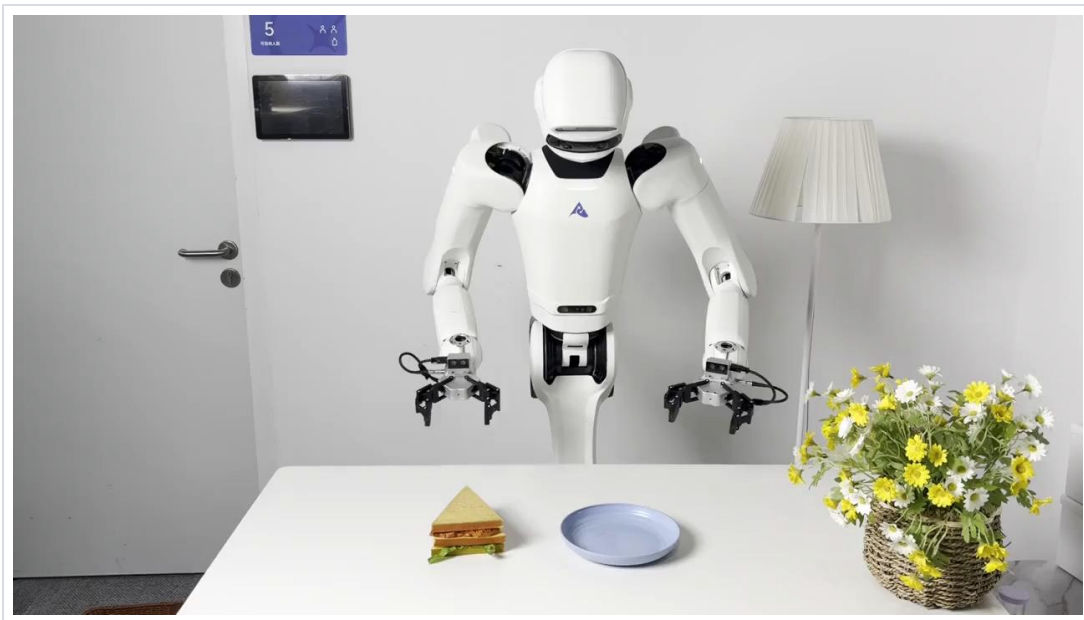
SIMPLER	Method	Success Rate(%) ↑					FLOPs (%)↓	Speed up↑
		Pick	Move	Drawer	Apple	Average		
Visual Matching	CogACT	91.3	85.0	71.8	50.9	74.8	100.0	1.00×
	Random	9.7	20.4	53.5	0.0	20.9	58.5	1.20×
	FastV	92.6	81.4	69.8	52.4	74.1	42.0	1.21×
	VLA-Cache	92.0	83.3	70.5	51.6	74.4	80.1	1.38×
	Ours	91.3	83.8	70.8	51.9	74.5	37.4	1.73×
Variant Aggregation	CogACT	89.6	80.8	28.3	46.6	61.3	100.0	1.00×
	Random	4.0	16.1	15.6	0.0	8.9	58.5	1.20×
	FastV	91.4	78.6	27.6	50.6	62.1	42.0	1.19×
	VLA-Cache	91.7	79.3	32.5	45.8	62.3	82.6	1.37×
	Ours	92.1	79.2	28.0	48.1	61.9	36.2	1.67×

03 • REAL ROBOT

Method	Success Rate ↑				FLOPs (T)↓	Latency (ms)↓
	<i>Pick and Place</i>	<i>Throw Basketball</i>	<i>Pack Doll</i>	<i>Average</i>		
$\pi_{0.5}$	94%	77%	73%	81.3%	2.264	80.36
+ Fast V	82%	69%	56%	69.0%	1.857	56.93
+ Ours	90%	81%	67%	79.3%	1.543	43.56

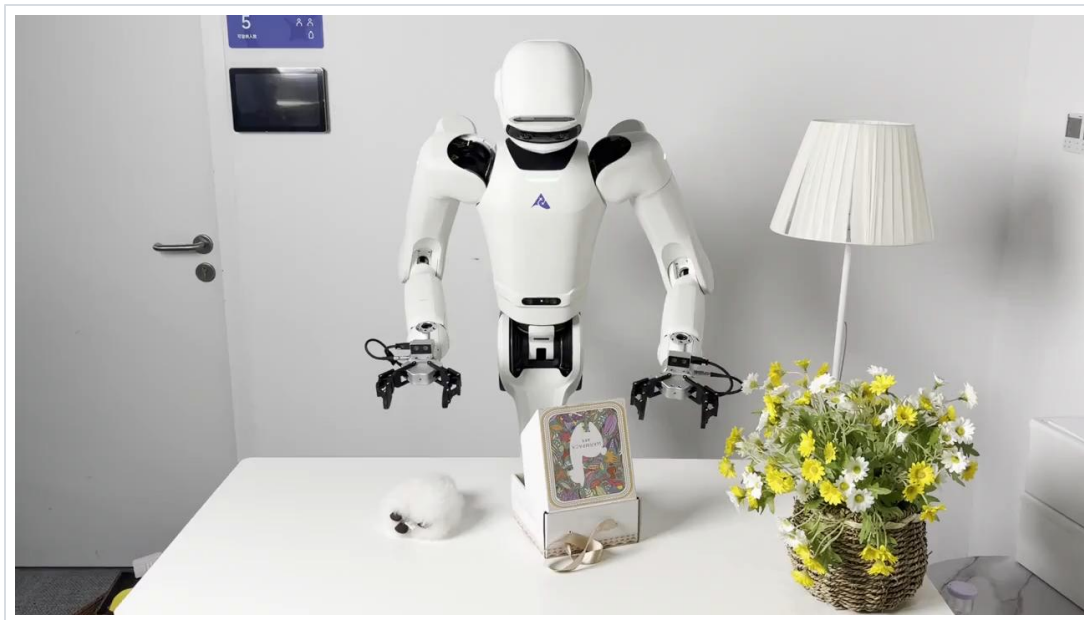
真实机器人 Rollouts

PICK & PLACE



抓取 → 对齐 → 放置

PACK DOLL



定位 → 抓取 → 重定位 → 装盒

结论：两项任务均保持动作连续性，未见 token 剪枝导致的明显停顿

SAFE-Pruner + HiF8: 成功率与 FLOPs 基本不变, 显存降低 38.1%

GPU MEMORY / GB



方法	Avg SR	FLOPs
OpenVLA-OFT	96.8	3.970T
SAFE-Pruner	96.4	1.722T
SAFE-Pruner + HiF8	96.6	1.729T

SAFE-Pruner 负责减少计算量和推理时延
HiF8 负责降低GPU显存占用

FLOPs -56.4% • GPU Memory -38.1% • Avg Success 96.6%

目录

Part 1 背景

Part 2 训练时计算

Part 3 推理时计算

Part 4 Safe-Pruner+HiFloat8

HiF8数据格式

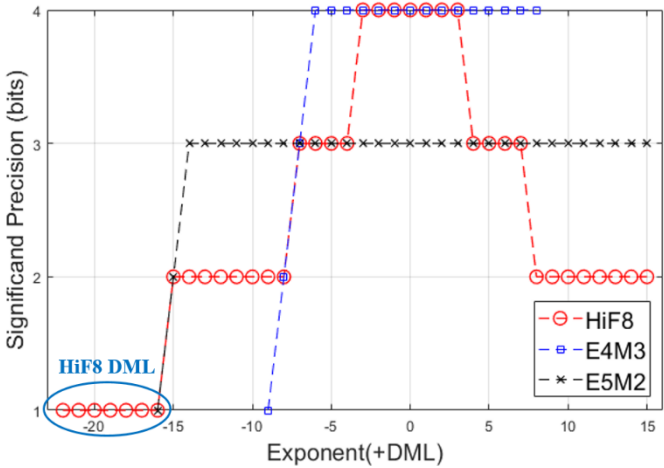
① 兼顾精度和动态范围



HiF8介绍:

- HiF8利用即时可译的变长前缀码编码点位域Dot, 显示指示阶码存储的位宽和Denormal标志, 实现同时符合AI训练和推理数据分布特征的锥形精度格式。
- 同时阶码采用原码编码, 并隐藏了1-bit固定值不存储, 确保了不同位宽的阶码表达范围不重复, 进而实现无冗余编码。
- 通过特殊的Denormal设计, 将综合阶码范围从[-15, 15]提升到了[-22, 15], 共38个阶码, 接近FP16的40个阶码值表达。
- HiF8编码了4个特殊值 ($\pm\text{Inf}$, Zero, NaN), 不区分正负0, 是一个表达完备的8-bit单数据浮点格式

② 无冗余编码: 锥形精度阶码图



HiF8精度区间严格不重叠:

- 高精度区间: $E = [-3, 3]$, 7个二进制指数, 4-bit有效位
- 中精度区间: $E = \pm[4, 7]$, 8个二进制指数, 3-bit有效位
- 低精度区间: $E = \pm[8, 15]$, 16个二进制指数, 2-bit有效位
- 极小值区间: $E = [-22, -16]$, 7个二进制指数, 1-bit有效位

③ 表达完备的8-bit单数据浮点格式

	FP16	HiF8	FP8-E4M3	FP8-E5M2
Infinites	Support	$S110111_2$	N/A	Support
NaN	Support	1000000_2	Support	Support
Zero	Support	0000000_2	Support	Support
Max Positive NML	$2^{15} \times (2 - 2^{-10})$	$01101110_2 = 2^{15}$	1.75×2^8	1.75×2^{15}
Min Positive NML	2^{-14}	$01111110_2 = 2^{-15}$	2^{-6}	2^{-14}
Max Positive DML	$2^{-14} \times (1 - 2^{-10})$	$00000111_2 = 2^{-16}$	1.75×2^{-7}	1.5×2^{-15}
Min Positive DML	2^{-24}	$00000001_2 = 2^{-22}$	2^{-9}	2^{-16}
Exponent	[-14, 15]	[-15, 15]	[-6, 8]	[-14, 15]
Exponent (+DML)	[-24, 15]	[-22, 15]	[-9, 8]	[-16, 15]

特殊值支持:

- $S = 0, D = \text{DML}, M = 3'$ b0, $X = \text{Zero}$ (HiF8: $8'$ b0000000)
- $S = 1, D = \text{DML}, M = 3'$ b0, $X = \text{NaN}$ (HiF8: $8'$ b10000000)
- $D = 4, E_s = 4'$ b0111 = 15, $M = 1'$ b1时, $X = \pm \text{Inf}$ (HiF8: $8'$ bX1101111)



综上, HiF8兼顾精度和动态范围, 为神经网络的训练和推理, 提供了能力更全面的8-bit单数据浮点格式表达

[1] Luo, Yuanyong, et al. "Ascend hifloat8 format for deep learning." arXiv preprint arXiv:2409.16626 (2024) <https://arxiv.org/abs/2409.16626> <https://ascend.com/cann>

为什么剪枝后的 VLA 特别难量化

三重不利条件叠加：量化格式不仅决定输出精度，也决定剪枝逻辑本身的可靠性

01

动作生成路径对精度高度敏感

输出为连续控制信号（回归 / 扩散 / flow-matching 动作头）。微小数值误差直接转化为不准确动作，并在闭环控制中逐步累积。相比语言任务中可被吸收的 logits 扰动，VLA 的容错空间小得多。

02

剪枝后剩余的都是任务关键 token

误差稀释效应消失了；剪枝和量化消耗的是同一份容错余量，剪枝留下的恰好是激活幅值最大、最难量化的长尾；而冗余副本被删后，关键信息从分布式表示退化成单点依赖。

Attention 输出 $o = \sum_i a_i v_i, \sum_i a_i = 1$

$$\mathbb{E} \|\tilde{o} - o\|^2 = \sigma^2 d \sum_i a_i^2 = \frac{\sigma^2 d}{N_{\text{eff}}}, \quad N_{\text{eff}} \triangleq \frac{1}{\sum_i a_i^2}$$

N_{eff} 是有效参与 token 数(逆参与比)

$$\text{量化误差幅度} \propto \frac{1}{\sqrt{N_{\text{eff}}}}$$

从 $N_{\text{eff}} = 1000$ 压到 $N_{\text{eff}} = 10$ ，量化误差在输出上的幅度放大约 10 倍

03

量化误差会与剪枝误差相互叠加

剪枝改变 softmax 的归一化分母(在剩余 token 上重新归一化)，量化改变 logits 的数值。两者都在扰动同一个 attention 分布。

量化误差 $\rightarrow$ saliency 排序错乱 $\rightarrow$ 剪错 token $\rightarrow$ 后续层分布偏移 $\rightarrow$ 更大的量化误差

双向恶化,不是单向:

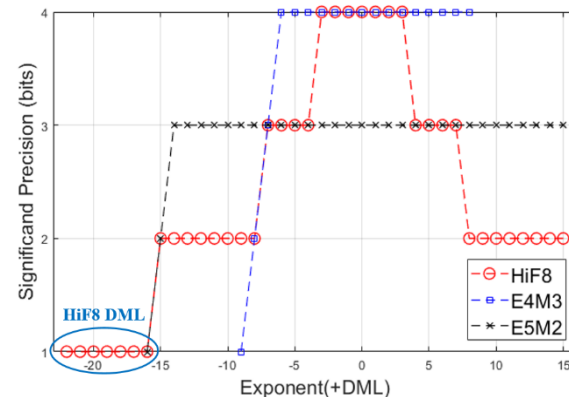
- 剪枝 $\rightarrow$ 保留下来的都是 outlier-rich 的 token $\rightarrow$ 量化变难
- 量化 $\rightarrow$ saliency 估计不准 $\rightarrow$ 剪枝判断变不可靠

HiF8 为何恰好契合

设计目标与该场景的三重挑战高度一致

● 锥形精度

按数值大小动态分配指数位 / 尾数位，把表示能力集中在分布密集的中心区域



● 宽动态范围

结合 denormal 编码覆盖约 38 个 binades，接近 FP16 的 40 个，避免深层 attention 的 overflow / underflow

● 无需细粒度 rescaling

不依赖 per-token / per-block scale，单一 per-tensor scale 即可完成有效量化

● attention 数值保真

不显著扭曲 attention 分布，因而不破坏依赖 attention 的剪枝决策

Safe-Pruner x HiF8 量化

Ascend 950PR端到端实测

	FLOPs	Time (ms per policy)
OpenVLA-OFT-7B	3.970	88.9
SafePrune-FP16	1.722	81.7 (8.8% improved)
SafePrune-HiF8	1.722	70.2 (26.6% improved)

HiF8 的量化落点

- Q/K/V projections
- output projection
- FFN 中的 gate/up/down projections

对于最精度敏感的组件，即vision encoder和action head，则保持在相对更高精度

LIBERO	Vanilla (BF16)	Safe-Pruner (BF16)	Safe-Pruner + HiF8 (VLM-only)	Safe-Pruner + HiF8 (VLM+Action head)
Spatial (%)	98.4	98.0	98.4	98.4
Object (%)	98.2	98.0	98.4	97.8
Goal (%)	96.4	96.2	95.8	90.0
Long (%)	94.2	93.4	93.4	89.2
Average (%)	96.8	96.4	96.5	93.9
FLOPs (T)	3.970	1.722	1.722	1.722

- Safe-Pruner 在HiF8量化后可以**保持BF16精度**。
- 在Ascend 950PR上，端到端实测，相对BF16提速**26.6%**，还在持续优化中。

锥形精度高精度捕捉剪枝后保留的关键 token 与精度敏感的动作信号；宽动态范围适配深层 attention 的宽分布，使 attention saliency 不被扭曲，从而不破坏 future-saliency prediction 与 subtask-boundary detection。两条正交加速路径可以叠加而互不损害精度。

具生智能CANN社区

CANN

CANN/cann-recipes-embodied-ai

DeepSeek Harness 开源

+ 新建

99+

G

代码

Issues 9

Pull Requests 10

Actions

项目讨论

分析

Watch 1

Star 269

Fork 61

master

分支 1

Tags 0

搜索文件

CANNLab

下载zip

Clone

Larry_liuyi update actions runs-on config 0c80b4a6 创建于 2 天前 88次提交

文件	最后提交记录	最后更新时间
.gitcode	update actions runs-on config Co-authored-by: liuyi<larry.liuyi@huawei...	2 天前
3d_vision	[feat] 更新vggt一站式平台拉起脚本，以解决下载权重时HF CAS问题 Co-...	20 天前
docs	[refactor] 对3d_vision VGGT模型启动和优化参数yaml配置化 Co-authore...	1 个月前
locomotion	[feat] 新增昇腾服务器上NMR非线性机器人控制模型推理与训练 Co-auth...	2 个月前
manipulation	[fix] 0728 修复readme的描述以及FP16下mask溢出为-inf并导致输出NaN...	12 天前
module	[feat] add Hunyuan3D model Co-authored-by: goodmani3<juchanghao...	3 个月前
navigation	[docs] 修改仓库名字为 cann-recipes-embodied-ai Co-authored-by: che...	2 个月前
ops	[feat] 迁入 3DGS 训推样例 Co-authored-by: xieyajun<xieyajun1@huawe...	2 个月前
world_model	update: 更新文件 README_en.md Co-authored-by: m292<1056321632...	10 天前
gitattributes	[feat] 新增cosmos3昇腾适配样例 Co-authored-by: nzalan<ningzichuan...	1 个月前

项目介绍

本项目针对具身智能业务中的典型模型、加速算法，提供基于CANN平台的优化样例

Apache-2.0 Python 88 提交数

定制我的领域

README

规则集

举报项目

下载使用量

27,368

项目总下载次数（含Clone、Pull、zip包及release下载），每日凌晨更新

项目健康度



Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：以开源驱动技术创新，降低开发门槛，聚力构筑高性能
昇腾 AI 算力底座

Vision: Building an Open, Easy-to-Use, and Technology-leading AI
Computing Ecosystem

Mission: Drive technological innovation through open source, lower
development barriers, and jointly build a high-performance Ascend AI
computing infrastructure.



上CANN社区获取干货



关注CANN公众号获取资讯

<https://gitcode.com/cann>

CANN